Chapter 8 Command and Natural Languages

8.1 Introduction

- The basic goals of language design are
  - Precision
  - Compactness
  - Ease in writing and reading
  - Completeness
  - Speed in earning
  - Simplicity to reduce errors
  - Ease of retention over time

- Higher level goals include
  - Close correspondence between reality and the notation
  - Convenience in carrying out manipulations relevant to users' tasks
  - Compatibility with existing notations
  - Flexibility to accommodate novice and expert users
  - Expressiveness to encourage creativity
  - Visual appeal

- Constraints on a language include
  - Capacity for human beings to record the notation
  - The match between the recording and the display media
  - The convenience in speaking

- Computer had only a modest influence on spoken natural languages, compared to its enormous impact as a stimulus to the development of numerous new formal written languages. Early computers were built to perform mathematical computations, so the first programming languages had a strong mathematical flavor.
- Computer programming languages such as FORTRAN, COBOL, ALGOL, PL/I, and Pascal that were developed in the 1960's and early 1970's were designed for use in a noninteractive computer environment. Programmers would compose thousands of lines of code, carefully check that code, and then compile or interprest it by computer to produce a desired result. Incremental programming was one of the design considerations in BASIC and in advanced languages such as LISP, APL, and PROLOG.
- Scripting languages emphasizing screen presentation and mouse control became popular in the 1980's with the appearance of HyperCard, SuperCard, ToolBook, and so on. These languages included novel operators, such as ON MOUSEDOWN, BLINK, or IF FIRST CHARACTER OF THE MESSAGE BOX IS 'A'.
- Command languages, which originated with operating systems commands, are distinguished by their immediacy and by their impact on devices or information.

Users issues a command and watch what happens. If the result is correct, the next command is issued; if not, some other strategy is adopted

- Command languages are distinguished from menu selection systems in that their users must recall notation and initiate action.
- Command languages may consist of single commands or may have complex syntax.

8.2 Functionality to Support Users' Tasks

- People use computers and command language systems to accomplish a wide range of work, such as text editing, operating system control, bibliographic retrieval, database manipulation, electronic mail, financial management, airline or hotel reservations, inventory, manufacturing process control, and adventure games.
- A common design error is to provide an excessive numbers of objects and actions, which can overwhelm the user. Excessive objects and actions take more code to maintain, potentially cause more bugs, possibly incur slower execution, and require more help screens, error messages, and user manuals.
- At an early stage, the destructive actions – such as deleting objects or changing formats – should be evaluated carefully to ensure that they are reversible, or at least are protected from accidental invocation. Designers should also identify error conditions and prepare error messages.

8.3 Command Organization Strategies

- Several strategies for command organization have emerged. A unifying interface concept or metaphor aids learning, problem solving and retention.
- Designers often err by choosing a metaphor closer to the computer domain than to the user's task domain. Of course, metaphors can mislead the user, but careful design can reap the benefits while reducing the detriments.
- In a simple command set, each command is chosen to carry out a single task, and the number of commands matches the number of tasks.
- The second option is to follow each command that indicate objects to be manipulated. The commands may be separated from the arguments by a blank or other delimiter, and the arguments may have blanks or delimiters between them.
- Commands may have options to indicate special cases. The number of arguments, of options, and of permissible syntactic forms can grow rapidly.
- The Unix command language system is widely used, in spite of the complexity of its command formats, which have been critized severely. Here again, users will master complexity to benefit from he rich functionality in a system
- In a hierarchical command structure, the full set of commands is organized into a tree structure, like a menu
- If a hierarchical structure can be found for a set of tasks, it offers a meaningful structure to a large number of commands.

8.4 The Benefits of Structure

- Human learning, problem soling, and memory are realty facilitated by meaningful structure. If command languages are well designed, users can recognize the structure and can easily encode it in their semantic knowledge storage.
- Meaningful structure is beneficial for task concepts, computer concepts, and syntactic details of command languages. Yet many systems fail to provide a meaningful structure. Users of one operating system display information with the LIST, QUERY, HELP and TYPE commands, and move objects with the PRINT, TYPE, SPOOL SEND COPY or MOVE commands.
- An explicit list of design conventions in a guidelines document can be an aid to designers and managers.
- Several studies have shown that there are benefits associated with using a consistent order for arguments.
- Evidence that command structure affects performance comes from a comparison of 15 commands in a commercially used symbol oriented text editor, and revised commands that had a more keyword oriented style.
- Commands could be hierarchical or nonhierarchical and congruent or noncongruent.
- Memory and problem solving tasks showed that congruent forms were clearly superior and that the hierarchical forms were superior for several dependent measures. Error rates were dramatically lower for the congruent hierarchical forms.
- In summary, sources of structure that have proved advantageous include these:
    - Positional consistency
    - Grammatical consistency
    - Congruent pairing
    - Hierarchical form

8.5 Naming and Abbreviations

- Names are important for learning, problem solving and retention over time. When it contains only a few names, command set is relatively easy to master; but when it contained hundreds of names, the choice of meaningful, organization sets of names becomes more important. Similar results were found for programming tasks, where variable name choices were less important in small modules with from 10 to 20 names than in longer modules with dozens or hundreds of names
- One naming rule debate revolves around the question of specify versus generality. Specific terms can be more descriptive than general ones are, and if they are more distinctive, they may be more memorable. General terms may be more familiar and therefore easier to accept.
- Even thought command names should be meaningful for human learning, problem solving, and retention, they must satisfy another important criterion. They must be in harmony with the mechanism for expressing the commands to the computer.
- Commands requiring shifted keys or CTRL keys, special characters, or difficult to type sequences are likely to cause higher error rates.

- Novices who were required to use full command names before being taught 2 letter abbreviations made fewer errors with the abbreviations than did those who were taught the abbreviations from the start.
- Several studies support the notion that abbreviation should be accomplished by a consistent strategy. Here are 6 potential strategies.
    - Simple truncation
    - Vowel drop with simple truncation
    - First and final letter
    - First letter of each word in a phrase
    - Standard abbreviations fro other contexts
    - Phonics

- Truncation appears to be the most effective mechanism overall, but it has it's problems. Conflicting abbreviations appear often, and decoding an unfamiliar abbreviation may not be easy as when vowel dropping is used.
- Ehrenreich and Porcu offer a compromise set if guidelines

8.6 Command Menus

- To relieve the burden of memorization of commands, some designers offer users brief prompts of available commands, in a format called a command menu.
- Experienced users come to know the commands and do not need to read the prompt or the help screens. Intermittent users know the concepts and refer to the prompt to jog their memory and to get help in retaining the syntax for future uses.
- The prompting approach emphasizes syntax and serves frequent users.
- In command menus, users can use the mouse or arrow keys to highlight their choices, or can type single letter choices, but frequently users do not even look at the menus as they type sequences of 2, 3 4 or more single letters that come to be thought of as a command.

8.7 Natural Language in Computing

- Researchers hope to fulfill the Star Trek scenario in which computers will respond to commands users issues by speaking in natural language. Natural Language Interaction (NLI) might be defined as the operation of computers by people using familiar natural language to give instructions and receive responses.
- Problems with NLI lie in not only implementation on the computer, but also desirability for large numbers of users for a wide variety of tasks.
- Most designs for NLI do not provide information about task actions and objects; users are usually presented with a simple prompt that invites a natural language statement.
- Some NLI work turned to automatic speech recognition and speech generation to reduce the barriers to acceptance. There is some advantage to use of these technologies, but the results are still meager. A promising application is the selection of painting told by discrete word recognition thus elimating the

frustration and delay of moving the cursor from the object to the tool menu on the border and back again.

- Since general interaction is difficult to support, some designers have pursued a morel-limited goal of natural language queries (NLQ) against relational databases. The relational schema contains attribute names and the database contains attribute values, both f which are helpful in disambiguating queries.
- Supporters of NLQ can point with some pride at the modest success of the INTELLECT system, which had approximately 400 installations on large mainframe computers during the 1980's/. The system's appeal has faded in recent years as users have turned to other approaches.
- Several innovative implementation ideas helped to make INTELLECT appealing. First the parser used the contents of the database to parse queries. Second the system administrator could conveniently include guidance for handling domain specific requests, by indicating fields related to who, what, where, when, why and how queries. Third INTELLECT rephrased the user's query and display a response, such as PRINT THE CHECK NUMBERS WITH PAYEE = GEORGE BUSH.
- Text database searching is a growing application for natural language enthusiasts who have developed filters and parsers for queries expressed in natural language. At one end of the spectrum is the full understanding of the meaning of a query and fulfillment of the users information needs.
- Although the artificial intelligence community often frowns on natural language test generation (NLTG) this modest application does seem to be a worthy on. It handles certain simple tasks, such as the preparation of structured weather reports from complex mathematical models.
- More elaborate applications of NLTG include preparation of reports of medical laboratory of psychological tests. The computer generates not only readable reports, but also warnings or recommendations.
- Natural language interaction techniques have enjoyed a notable and widespread success in a variety of adventure games. Users may indicate directions of movement or type commands.

8.8 Practitioner's Summary

- Command languages can be attractive when frequent use of a system is anticipated, users are knowledgeable about the task and interface concepts, screen space is at a premium, response time and display rates are slow, and numerous functions can be combined in a computer expression.
- Designers should begin with a careful task analysis to determine what functions should be provided.

8.9 Researcher Agenda

- The benefits of structuring command languages based on hierarchy, congruence, consistency, and mnemonicity have been demonstrated in specific cases, but replication in varied situations should lead to a comprehensive cognitive model of command language learning and use.